

Doc 7.3 — Object Destructuring & Property Shortcuts

Object নিয়ে কাজ করার সময় বারবার এমন code লিখতে হয়:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  email: "shariar@example.com"  
};  
  
const name = user.name;  
const age = user.age;  
const email = user.email;
```

JavaScript আমাদের এটাকে অনেক cleaner way-তে লেখার সুযোগ দেয়:

```
const { name, age, email } = user;
```

এটাই **Object Destructuring**!

এই Doc-এ আমরা destructuring-এর পাশাপাশি renaming, defaults, nested destructuring, function parameters, rest pattern, shorthand properties এবং shorthand methods শিখব।

1. Object Destructuring কী?

Object destructuring হলো object-এর property থেকে value বের করে সরাসরি variable-এ assign করার syntax।

Example:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const { name, age } = user;  
  
console.log(name);  
console.log(age);
```

Output:

```
Shariar
  31
```

2. Normal Property Access বনাম Destructuring

Normal way

```
const user = {
  name: "Shariar",
  age: 31
};

const name = user.name;
const age = user.age;
```

Destructuring

```
const user = {
  name: "Shariar",
  age: 31
};

const { name, age } = user;
```

দুই ক্ষেত্রেই একই data পাওয়া যাচ্ছে।

Destructuring শুধু syntax-কে concise করে।

3. Destructuring Position-Based নয়

Array destructuring position-based:

```
const [first, second] = [10, 20];
```

কিন্তু object destructuring **property name-based**।

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const { age, name } = user;
```

Order change করলেও সমস্যা নেই।

```
name → name property  
age → age property
```

4. Property Name Match করে

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const { name } = user;
```

এখানে:

```
object-এর name property  
    ↓  
name variable
```

5. শুধু একটি Property Destructure

সব property নিতে হবে এমন নয়।

```
const user = {
  name: "Shariar",
  age: 31,
  email: "shariar@example.com"
};

const { name } = user;

console.log(name);
```

Output:

```
Shariar
```

`age` এবং `email` destructure করা হয়নি।

6. Multiple Properties Destructure

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};

const { name, age, role } = user;
```

এখন:

```
console.log(name);
console.log(age);
console.log(role);
```

7. Destructuring-এ Variable-এর নাম Property-এর মতোই হয়

```
const user = {  
  name: "Shariar"  
};  
  
const { name } = user;
```

এখানে variable-এর নাম:

```
name
```

কারণ property-এর নামও:

```
name
```

8. Renaming — Property-এর নাম পরিবর্তন না করে Variable-এর নাম পরিবর্তন

ধরো:

```
const user = {  
  name: "Shariar"  
};
```

তুমি variable-এর নাম `userName` রাখতে চাও।

Syntax:

```
const { name: userName } = user;
```

এখন:

```
console.log(userName);
```

Output:

Shariar

9. Renaming Mental Model

এটা:

```
const { name: userName } = user;
```

দেখে beginner অনেক সময় ভাবে `name` থেকে `userName` property rename হচ্ছে।

আসলে তা নয়।

এটা বলছে:

```
object-এর name property
      ↓
    userName variable
```

Original object unchanged।

10. Renaming Example

```
const user = {
  name: "Shariar",
  age: 31
};

const {
  name: userName,
  age: userAge
} = user;

console.log(userName);
console.log(userAge);
```

Output:

```
Shariar
```

```
31
```

11. Renaming-এর Practical Use

ধরো API response:

```
const response = {  
  user_name: "Shariar",  
  user_age: 31  
};
```

তুমি JavaScript code-এ cleaner variable name চাও:

```
const {  
  user_name: userName,  
  user_age: userAge  
} = response;
```

এখন:

```
console.log(userName);  
console.log(userAge);
```

12. Default Values

যদি object-এর property না থাকে, destructured variable-এর value সাধারণত:

```
undefined
```

হয়।

```
const user = {  
  name: "Shariar"  
};  
  
const { role } = user;  
  
console.log(role);
```

Output:

```
undefined
```

Default value দিতে পারি:

```
const { role = "user" } = user;  
  
console.log(role);
```

Output:

```
user
```

13. Default কখন কাজ করে?

Default value তখনই ব্যবহার হয় যখন property value:

```
undefined
```

হয়।

Example:

```
const user = {  
  role: undefined  
};  
  
const { role = "user" } = user;  
  
console.log(role);
```


Output:

```
user
```

14. `null` হলে Default কাজ করে না

```
const user = {  
  role: null  
};  
  
const { role = "user" } = user;  
  
console.log(role);
```

Output:

```
null
```

কারণ default value শূন্য `undefined`-এর ক্ষেত্রে trigger হয়।

15. Default + Renaming

দুটো একসাথে ব্যবহার করা যায়।

```
const user = {  
  name: "Shariar"  
};  
  
const {  
  role: userRole = "user"  
} = user;  
  
console.log(userRole);
```

Output:

```
user
```

Mental model:

```
role property
  ↓
userRole variable
  ↓
না থাকলে "user"
```

16. Destructuring Assignment

Declaration-এর সময়:

```
const { name } = user;
```

এটা সবচেয়ে common।

কিন্তু existing variable-এও destructuring assignment করা যায়।

```
let name;

const user = {
  name: "Shariar"
};

({ name } = user);

console.log(name);
```

এখানে parentheses প্রয়োজন হতে পারে, কারণ `{ }`-কে JavaScript block হিসেবে interpret করতে পারে।

17. Nested Object Destructuring

ধরো:

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka",  
    country: "Bangladesh"  
  }  
};
```

Normal access:

```
const city = user.address.city;  
const country = user.address.country;
```

Destructuring:

```
const {  
  address: { city, country }  
} = user;
```

এখন:

```
console.log(city);  
console.log(country);
```

Output:

```
Dhaka  
Bangladesh
```

18. Nested Destructuring Mental Model

এই object:

```
user
├── name
└── address
    ├── city
    └── country
```

এই destructuring:

```
const {
  address: { city, country }
} = user;
```

মানে:

```
user
  ↓
address
  ↓
city + country
```

19. Nested Destructuring with Parent Object

শুধু nested properties নয়, parent object-টাও variable হিসেবে রাখতে পারি।

```
const user = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

const {
  address,
  address: { city }
} = user;
```

এখন দুটোই পাওয়া যাবে:

```
console.log(address);  
console.log(city);
```

20. Nested Default Values

Nested property missing হলে default দেওয়া যায়।

```
const user = {  
  name: "Shariar"  
};  
  
const {  
  address = {}  
} = user;
```

এখন `address` undefined হবে না; `{}` হবে।

21. Function Parameter Destructuring

Object destructuring-এর সবচেয়ে important practical use হলো function parameters।

ধরো:

```
function greet(user) {  
  console.log(user.name);  
  console.log(user.age);  
}
```

এটা destructuring দিয়ে:

```
function greet({ name, age }) {  
  console.log(name);  
  console.log(age);  
}
```

Call:

```
greet({
  name: "Shariar",
  age: 31
});
```

22. কেন Function Parameter Destructuring Useful?

Normal:

```
function createUser(user) {
  console.log(user.name);
  console.log(user.email);
  console.log(user.role);
}
```

Destructuring:

```
function createUser({ name, email, role }) {
  console.log(name);
  console.log(email);
  console.log(role);
}
```

Function-এর ভিতরের code cleaner হয়।

23. Function Parameter Default

```
function greet({ name = "Guest" }) {
  console.log(`Hello ${name}`);
}
```

Call:

```
greet({});
```

Output:

```
Hello Guest
```

24. Function Parameter Object Default

একটা গুরুত্বপূর্ণ difference:

```
function greet({ name = "Guest" }) {  
  console.log(name);  
}
```

এখানে:

```
greet();
```

করলে error হতে পারে, কারণ parameter নিজেই `undefined`।

Safe version:

```
function greet({ name = "Guest" } = {}) {  
  console.log(name);  
}
```

এখন:

```
greet();
```

Output:

```
Guest
```

25. React Component Props Destructuring

React-এ এটি খুব common:

```
function UserCard({ name, age }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>{age}</p>  
    </div>  
  );  
}
```

Props object:

```
props  
├─ name  
└─ age
```

Function parameter-এই destructure হয়ে যাচ্ছে।

26. React-এর Normal Props বনাম Destructuring

Normal:

```
function UserCard(props) {  
  return (  
    <div>  
      <h2>{props.name}</h2>  
      <p>{props.age}</p>  
    </div>  
  );  
}
```

Destructuring:


```
function UserCard({ name, age }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>{age}</p>  
    </div>  
  );  
}
```

দুটোই valid।

27. Destructuring Function Return

Function object return করতে পারে:

```
function getUser() {  
  return {  
    name: "Shariar",  
    age: 31  
  };  
}
```

Normal:

```
const user = getUser();  
  
console.log(user.name);
```

Destructuring:

```
const { name, age } = getUser();  
  
console.log(name);  
console.log(age);
```

এটা real-world code-এ খুব common।

28. Rest Pattern in Object Destructuring

Object destructuring-এর মধ্যে `...` ব্যবহার করে remaining properties collect করা যায়।

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer",
  city: "Dhaka"
};

const { name, ...others } = user;

console.log(name);
console.log(others);
```

Output:

```
Shariar

{
  age: 31,
  role: "Developer",
  city: "Dhaka"
}
```

29. Rest Pattern Mental Model

```
const {
  name,
  ...others
} = user;
```

মানে:

name → আলাদা variable

বাকি সব properties

↓

others object

30. Rest Pattern vs Spread

দুটোতেই `...` দেখা যায়, কিন্তু কাজ আলাদা।

Spread

```
const newUser = {  
  ...user  
};
```

Existing values **spread/copy** করে।

Rest

```
const { name, ...others } = user;
```

Remaining values **collect** করে।

সহজভাবে:

Spread → ছড়িয়ে দেয়

Rest → বাকি অংশ জড়ো করে

31. Rest দিয়ে Property Remove Pattern

আগের Doc-এর immutable removal pattern:

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};

const { role, ...userWithoutRole } = user;
```

এখন:

```
console.log(userWithoutRole);
```

Result:

```
{
  name: "Shariar",
  age: 31
}
```

Original object unchanged!

32. Multiple Properties বাদ দিয়ে Rest

```
const user = {
  name: "Shariar",
  age: 31,
  email: "shariar@example.com",
  role: "Developer"
};

const {
  email,
  role,
  ...basicInfo
} = user;
```

`basicInfo`:

```
{  
  name: "Shariar",  
  age: 31  
}
```

33. Object Shorthand Property Names

ধরো:

```
const name = "Shariar";  
const age = 31;
```

Normal object:

```
const user = {  
  name: name,  
  age: age  
};
```

JavaScript shorthand syntax দেয়:

```
const user = {  
  name,  
  age  
};
```

এটা একই result দেয়।

34. Shorthand Mental Model

যদি:

```
const name = "Shariar";
```

তাহলে:

```
const user = {  
  name  
};
```

মানে:

```
const user = {  
  name: name  
};
```

35. Multiple Shorthand Properties

```
const name = "Shariar";  
const age = 31;  
const role = "Developer";  
  
const user = {  
  name,  
  age,  
  role  
};
```

Equivalent:

```
const user = {  
  name: name,  
  age: age,  
  role: role  
};
```

36. Shorthand-এর Practical Use

React/API code-এ এটা খুব common:

```
const name = formData.name;
const email = formData.email;
const age = formData.age;

const user = {
  name,
  email,
  age
};
```

Clean এবং readable।

37. Shorthand Method Syntax

Object-এর ভিতরে function define করারও shorthand syntax আছে।

পুরনো style:

```
const user = {
  greet: function () {
    console.log("Hello");
  }
};
```

Modern shorthand:

```
const user = {
  greet() {
    console.log("Hello");
  }
};
```

দুটোই method তৈরি করে।

38. Method Call

```
const user = {  
  name: "Shariar",  
  
  greet() {  
    console.log(`Hello ${this.name}`);  
  }  
};  
  
user.greet();
```

Output:

```
Hello Shariar
```

এখানে `greet()` হলো object-এর method।

39. Shorthand Property + Shorthand Method একসাথে

```
const name = "Shariar";  
  
const user = {  
  name,  
  
  greet() {  
    console.log(`Hello ${this.name}`);  
  }  
};
```

এখানে:

```
name → shorthand property  
greet() → shorthand method
```


40. Destructuring + Shorthand-এর Relationship

এই দুই syntax অনেক সময় একসাথে দেখা যায়।

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const { name, age } = user;  
  
const profile = {  
  name,  
  age  
};
```

এখানে:

Object → destructure → variables → shorthand object

41. Destructuring-এর সাথে Variable Name Conflict

ধরো:

```
const name = "John";  
  
const user = {  
  name: "Shariar"  
};  
  
const { name } = user;
```

এটা error হবে, কারণ একই scope-এ `name` variable already declared।

এক্ষেত্রে rename করো:

```
const name = "John";

const user = {
  name: "Shariar"
};

const { name: userName } = user;
```

42. Default + Existing Variable

```
const user = {
  name: "Shariar"
};

const {
  role = "developer"
} = user;
```

Result:

```
role = "developer"
```

কিন্তু যদি:

```
const user = {
  role: "admin"
};

const {
  role = "developer"
} = user;
```

তাহলে:

```
role = "admin"
```

Default existing value overwrite করে না।

43. Renaming + Default + Nested

সব একসাথে করা সম্ভব।

```
const user = {
  name: "Shariar",
  address: {
    city: "Dhaka"
  }
};

const {
  name: userName,
  role: userRole = "user",
  address: {
    city: userCity = "Unknown"
  }
} = user;
```

এখন:

```
userName → "Shariar"
userRole → "user"
userCity → "Dhaka"
```

44. Destructuring API Response

API response সাধারণত:

```
const response = {
  data: {
    user: {
      id: 101,
      name: "Shariar"
    }
  },
  status: 200
};
```

Normal:

```
const user = response.data.user;

console.log(user.name);
```

Destructuring:

```
const {
  data: {
    user: { id, name }
  },
  status
} = response;
```

এখন:

```
console.log(id);
console.log(name);
console.log(status);
```

45. Practical React Example

```
function ProductCard({ name, price, image }) {
  return (
    <article>
      <img src={image} alt={name} />
      <h2>{name}</h2>
      <p>{price}</p>
    </article>
  );
}
```

এখানে props object destructure করা হচ্ছে function parameter-এই।

46. Practical Form Example

```
const formData = {
  firstName: "Shariar",
  lastName: "Mahamud",
  email: "shariar@example.com"
};

const {
  firstName,
  lastName,
  email
} = formData;

const user = {
  firstName,
  lastName,
  email
};
```

এখানে destructuring এবং shorthand দুটোই ব্যবহার হয়েছে।

47. Practical Update Example

ধরো:

```
const user = {
  name: "Shariar",
  age: 31,
  role: "Developer"
};
```

শুধু `name` এবং বাকি data আলাদা করতে:

```
const {
  name,
  ...details
} = user;
```

এখন:

```
name
  → "Shariar"

details
  → {
    age: 31,
    role: "Developer"
  }
```

48. Destructuring কি Object Mutate করে?

না।

```
const user = {
  name: "Shariar",
  age: 31
};

const { name } = user;
```

এখানে original object পরিবর্তন হয় না।

Destructuring শুধু values extract করে।

49. Destructuring কি Deep Copy?

না।

এটাও মনে রাখা জরুরি।

```
const user = {
  address: {
    city: "Dhaka"
  }
};

const { address } = user;
```

এখানে `address` হলো একই nested object-এর reference।

```
address.city = "Rangpur";

console.log(user.address.city);
```

Output:

```
Rangpur
```

Destructuring নিজে deep copy তৈরি করে না।

50. Common Mistake — Array Syntax ব্যবহার করা

Object-এর জন্ম:

```
const { name } = user;
```

Array-এর জন্ম:

```
const [first] = users;
```

Object destructuring-এ `{}` এবং array destructuring-এ `[]`।

51. Common Mistake — Property Name ভুল করা

```
const user = {
  name: "Shariar"
};

const { username } = user;

console.log(username);
```

Output:

```
undefined
```

কারণ object-এ `username` property নেই।

52. Common Mistake — Rename Syntax ভুল

ভুল:

```
const { name = userName } = user;
```

এটা default value বোঝাবে।

Rename করতে:

```
const { name: userName } = user;
```

53. Rename Syntax Mental Model

এইটা মনে রাখো:

```
const { originalName: newVariableName } = object;
```

Example:

```
const { name: userName } = user;
```

মানে:

```
object.name  
  ↓  
userName
```


54. Common Mistake — Default এবং Null

```
const user = {  
  name: null  
};  
  
const { name = "Guest" } = user;  
  
console.log(name);
```

Output:

```
null
```

কারণ default শুধু `undefined`-এর জন্য।

55. Common Mistake — Function Without Argument

এই function:

```
function greet({ name }) {  
  console.log(name);  
}
```

এখানে:

```
greet();
```

safe নয়।

Better:

```
function greet({ name } = {}) {  
  console.log(name);  
}
```

অথবা default:

```
function greet({ name = "Guest" } = {}) {  
  console.log(name);  
}
```

56. Common Mistake — Rest Pattern-এর পরে আর Property

ভুল:

```
const {  
  name,  
  ...others,  
  age  
} = user;
```

Rest element অবশ্যই destructuring pattern-এর শেষে থাকতে হবে।

সঠিক:

```
const {  
  name,  
  age,  
  ...others  
} = user;
```

57. Destructuring Quick Reference

Pattern	কাজ
<code>const { name } = user</code>	property extract
<code>const { name: userName } = user</code>	rename
<code>const { role = "user" } = user</code>	default
<code>const { name: userName = "Guest" } = user</code>	rename + default
<code>const { address: { city } } = user</code>	nested destructuring
<code>const { name, ...rest } = user</code>	remaining properties
<code>function fn({ name }) {}</code>	parameter destructuring
<code>const { name } = getUser()</code>	return value destructuring

58. Shorthand Quick Reference

Normal

```
const user = {
  name: name,
  age: age
};
```

Shorthand

```
const user = {
  name,
  age
};
```

59. Method Shorthand

Old style

```
const user = {
  greet: function () {
    console.log("Hello");
  }
};
```

Modern

```
const user = {
  greet() {
    console.log("Hello");
  }
};
```

60. Object Destructuring vs Normal Access

Task	Normal	Destructuring
One value	<code>user.name</code>	<code>const { name } = user</code>
Multiple values	<code>user.name</code> , <code>user.age</code>	<code>const { name, age } = user</code>
Rename	<code>const userName = user.name</code>	<code>const { name: userName } = user</code>
Default	<code>user.role ?? "user"</code>	<code>const { role = "user" } = user</code>
Nested	<code>user.address.city</code>	<code>const { address: { city } } = user</code>

61. Practice Questions

Question 1

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
const { name, age } = user;  
  
console.log(name);  
console.log(age);
```

Question 2

Output কী হবে?

```
const user = {  
  name: "Shariar"  
};  
  
const { name: userName } = user;  
  
console.log(userName);
```

Question 3

Output কী হবে?

```
const user = {};  
  
const { role = "user" } = user;  
  
console.log(role);
```

Question 4

Output কী হবে?

```
const user = {  
  role: null  
};  
  
const { role = "user" } = user;  
  
console.log(role);
```

Question 5

Output কী হবে?

```
const user = {  
  name: "Shariar",  
  age: 31,  
  role: "Developer"  
};  
  
const { name, ...others } = user;  
  
console.log(name);  
console.log(others);
```

Question 6

Output কী হবে?

```
const user = {  
  address: {  
    city: "Dhaka"  
  }  
};  
  
const {  
  address: { city }  
} = user;  
  
console.log(city);
```

Question 7

Output কী হবে?

```
const name = "Shariar";  
const age = 31;  
  
const user = {  
  name,  
  age  
};  
  
console.log(user);
```

Question 8

Output কী হবে?

```
function greet({ name = "Guest" } = {}) {  
  console.log(name);  
}  
  
greet();  
greet({});  
greet({ name: "Shariar" });
```

62. Coding Challenges

Challenge 1 — Rename Properties

এই object:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

Destructuring করে variables বানাও:

```
userName  
userAge
```

Challenge 2 — Default Role

```
const user = {  
  name: "Shariar"  
};
```

Destructuring ব্যবহার করে:

```
name  
role
```

extract করো।

`role` না থাকলে:

```
"User"
```

হবে।

Challenge 3 — Nested Destructuring

```
const user = {  
  name: "Shariar",  
  address: {  
    city: "Dhaka",  
    country: "Bangladesh"  
  }  
};
```

Destructuring ব্যবহার করে সরাসরি:

```
name  
city  
country
```

variables তৈরি করো।

Challenge 4 — Remove Property

```
const user = {  
  name: "Shariar",  
  age: 31,  
  email: "shariar@example.com"  
};
```

Destructuring + rest ব্যবহার করে `email` বাদ দিয়ে নতুন object তৈরি করো।

Challenge 5 — React Props

একটা React component তৈরি করো:

```
UserCard
```

যেটা props হিসেবে নেবে:

```
name
age
role
```

এবং destructuring ব্যবহার করে UI-তে render করবে।

63. Interview Questions

1. Object destructuring কী?

Object-এর property থেকে value extract করে সরাসরি variable-এ assign করার syntax।

2. Object destructuring কি position-based?

না।

Object destructuring property-name based।

3. Destructuring-এ rename কীভাবে করা হয়?

```
const { name: userName } = user;
```

4. Destructuring default কখন কাজ করে?

Property-এর value `undefined` হলে।

`null` হলে default কাজ করে না।

5. Object destructuring কি original object modify করে?

না।

6. Destructuring কি deep copy করে?

না।

Nested object হলে reference একই থাকতে পারে।

7. Rest pattern কী?

Destructuring-এর সময় remaining properties collect করার syntax।

```
const { name, ...rest } = user;
```

8. Spread এবং Rest-এর difference কী?

Spread → values spread করে

Rest → remaining values collect করে

9. Object shorthand কী?

যখন variable name এবং property name একই, তখন:

```
const user = {  
  name  
};
```

লেখা যায়, যা:

```
const user = {  
  name: name  
};
```

এর equivalent।

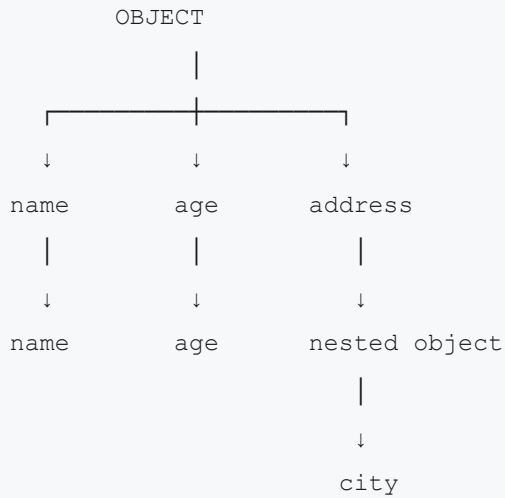
10. Function parameter destructuring কী?

Function parameter হিসেবে আসা object-এর properties সরাসরি parameter list-এ destructure করা।

```
function greet({ name }) {  
  console.log(name);  
}
```

64. Mental Model

Object destructuring-কে এভাবে চিন্তা করো:



Destructuring:

```
const {  
  name,  
  age,  
  address: { city }  
} = user;
```

Result:

```
name → "Shariar"  
age  → 31  
city → "Dhaka"
```

65. Full Object Syntax Mental Model

এখন পর্যন্ত Object-এর তিনটা বড় layer:

OBJECTS

- |
 - ├─ Doc 7.1
 - | Fundamentals & Access
 - | ─ {}
 - | ─ Properties
 - | ─ Dot notation
 - | ─ Bracket notation
 - | ─ Add
 - | ─ Update
 - | ─ Delete
 - | ─ Nested objects
 - | └─ References
 - ├─ Doc 7.2
 - | Methods & Copying
 - | ─ Object.keys()
 - | ─ Object.values()
 - | ─ Object.entries()
 - | ─ Object.assign()
 - | ─ Spread
 - | ─ Merge
 - | └─ Shallow copy
 - └─ Doc 7.3
 - | Destructuring & Shortcuts
 - | ─ Destructuring
 - | ─ Rename
 - | ─ Default
 - | ─ Nested
 - | ─ Function parameters
 - | ─ Rest pattern
 - | ─ Property shorthand
 - | └─ Method shorthand

66. Golden Rules

🏆 Rule 1 Object destructuring *হলো* **property-based extraction** /

🏆 Rule 2 `{ name }` মানে object-এর `name` property extract করা।

🏆 Rule 3 `{ name: userName }` মানে `name` property → `userName` variable।

🏆 Rule 4 Default value শুধু `undefined` হলে কাজ করে।

🏆 Rule 5 Nested destructuring দিয়ে deep property সরাসরি extract করা যায়।

🏆 Rule 6 Function parameters-এ object destructuring React-এ অত্যন্ত common।

🏆 Rule 7 `...rest` remaining properties collect করে।

🏆 Rule 8 Spread এবং Rest দেখতে একই হলেও কাজ আলাদা।

🏆 Rule 9 Shorthand property তখনই ব্যবহার করা যায় যখন variable এবং property name একই।

🏆 Rule 10 Destructuring নিজে object-এর deep copy তৈরি করে না।

67. Final Mastery Checklist

এই Doc শেষ করার পর তোমার এগুলো confidently পারা উচিত:

- ☐ Object destructuring বুঝি
- ☐ Multiple properties destructure করতে পারি
- ☐ Property rename করতে পারি
- ☐ Default value দিতে পারি
- ☐ Rename + default একসাথে করতে পারি
- ☐ Nested object destructure করতে পারি
- ☐ Nested default বুঝি
- ☐ Function parameter destructuring করতে পারি
- ☐ Function parameter-এর default object দিতে পারি
- ☐ Function return থেকে destructure করতে পারি
- ☐ Object rest pattern ব্যবহার করতে পারি
- ☐ Rest দিয়ে property বাদ দিতে পারি
- ☐ Spread বনাম Rest explain করতে পারি
- ☐ Property shorthand বুঝি
- ☐ Method shorthand বুঝি
- ☐ React props destructure করতে পারি

- ☐ API response destructure করতে পারি
- ☐ Destructuring deep copy নয়—এটা বুঝি
- ☐ Common destructuring mistakes avoid করতে পারি



Next Doc



Doc 7.4 — Computed Properties & Advanced Object Patterns

শেষ Object doc-এ আমরা দেখব:

Computed Property Names

↓

Dynamic Keys

↓

Dynamic Object Creation

↓

Form Data Patterns

↓

API Data Patterns

↓

Spread + Computed Properties

↓

Dynamic React State Update

↓

Advanced Object Patterns

বিশেষ করে এই pattern:

```
const key = "name";

const user = {
  [key]: "Shariar"
};
```

এবং React-এর খুব common pattern:

```
setFormData({  
  ...formData,  
  [name]: value  
});
```

এগুলো বুঝে গেলে JavaScript object manipulation-এর practical foundation অনেকটাই complete হয়ে যাবে।